QUESTION 1

Study the ARFF File Format and Explore Datasets in WEKA

Aim

To study the ARFF file format, explore the available datasets in WEKA, prepare an ARFF file and execute it on WEKA.

ARFF File Format

ARFF stands for Attribute-Relation File Format. It is an ASCII text file that describes a list of instances sharing a set of attributes. ARFF files have two sections: Header information and Data information.

```
@RELATION iris
@ATTRIBUTE sepallength NUMERIC
@ATTRIBUTE sepalwidth NUMERIC
@ATTRIBUTE petallength NUMERIC
@ATTRIBUTE petalwidth NUMERIC
@ATTRIBUTE class {Iris-setosa,Iris-versicolor,Iris-virginica}
@DATA
```

5.1,3.5,1.4,0.2,Iris-setosa

4.9,3.0,1.4,0.2,Iris-setosa

4.7,3.2,1.3,0.2,Iris-setosa

4.6,3.1,1.5,0.2,Iris-setosa

5.0,3.6,1.4,0.2,Iris-setosa

ARFF Header

The header contains the relation declaration and attribute declarations. Each attribute is declared using @attribute . WEKA supports numeric, integer, real, nominal, string and date attribute types.

Data Section

The data section starts with @data. Each instance is written on a separate line, with values separated by commas. Missing values are represented by ?. Lines beginning with % are comments.

Result

The ARFF file format was studied and the available datasets in WEKA were explored.

QUESTION 2

Apply Discretization and Run Apriori Association Rule Algorithm

Aim

To apply different discretization filters on numerical attributes and run the Apriori association rule algorithm and study the rules generated.

Procedure

1. Open WEKA and go to Explorer.
2. Load iris.arff.
3. Check the attribute types.
4. Since Iris attributes are numeric, apply discretization first.
5. Apply discretization to the required numerical attributes.
6. Go to the Association tab.
7. Select Apriori using Choose.
8. Set the required options.

9. Click Start to generate association rules.

Discretization

The original record demonstrates discretizing the duration attribute from labor.arff. Select the Discretize filter, enter the attribute index, set the number of bins to 10 and click Apply. The resulting dataset is saved as labor-data-discretized.arff.

```
weka.filters.unsupervised.attribute.Discretize
```

Apriori Options

The PDF lists minMetric, verbose, numRules, lowerBoundMinSupport, classIndex, outputItemSets, car, significanceLevel, delta and metricType.

Result

The numerical attributes were discretized and Apriori was successfully applied to generate association rules.

QUESTION 3

ID3 and J48 Classification Algorithm

Aim

To load datasets into WEKA and run ID3 and J48 classification algorithms, study classifier output and compute entropy values and Kappa statistic.

J48 Procedure

1. Open WEKA Explorer.
2. Load iris.arff.
3. Select Classify.
4. Click Choose and select J48.
5. Accept default parameters.
6. Select 10-fold cross-validation.
7. Click Start.
8. Observe the decision tree and evaluation statistics.
9. Use Visualize tree to view the graphical tree.

J48 Options

The PDF lists seed, unpruned, confidenceFactor, numFolds, numDecimalPlaces, batchSize, reducedErrorPruning, useLaplace, subtreeRaising, binarySplits, minNumObj and useMDLcorrection.

ID3 Procedure

1. Open WEKA Explorer.
2. Load weather.nominal.arff.
3. Check attributes and missing values.
4. Select Classify.
5. Select ID3.
6. Accept default parameters.
7. Select 10-fold cross-validation.
8. Click Start.
9. Observe the decision tree and evaluation statistics.

Result

ID3 and J48 classification algorithms were successfully executed in WEKA and the classifier outputs were studied.

QUESTION 4

Explore Other Clustering Techniques Available in WEKA

Aim

DWDM LAB — Original PDF Content — Page 3

To explore clustering techniques available in WEKA.

Simple K-Means

1. Open WEKA Explorer.
2. Load iris.arff.
3. Select Cluster.
4. Click Choose and select Simple K-Means.
5. Specify the parameters.
6. Set numClusters.
7. Set maxIterations.
8. Run the algorithm.
9. Observe clusters, centroids and sum of squared errors.

HierarchicalClusterer

```
weka.clusterers.HierarchicalClusterer
```

The PDF lists numClusters, linkType, distanceFunction and printNewick. Linkage methods include SINGLE, COMPLETE, ADJCOMPLETE, AVERAGE, MEAN, CENTROID, WARD and NEIGHBOR_JOINING.

Procedure

1. Open WEKA Explorer.
2. Load iris.arff.
3. Select Cluster.
4. Choose HierarchicalClusterer.
5. Specify parameters.
6. Select the cluster mode.
7. Select Classes to clusters evaluation.
8. Click Start.
9. Observe statistics and cluster assignments.
10. Visualize cluster assignments if required.

Result

Different clustering techniques available in WEKA were explored successfully.

QUESTION 5

Knowledge Flow Layout Using Apriori and FP-Growth

Aim

To develop a Knowledge Flow layout for finding strong association rules using Apriori and FP-Growth algorithms.

Knowledge Flow

Knowledge Flow provides a graphical front end to WEKA. WEKA steps can be selected from a palette, placed on a layout canvas and connected together. Classifiers, filters, clusterers, associators, loaders and savers are available.

ARFF Loader

|

v

Preprocessing

|

+---+---+

| |

v v

Apriori FP-Growth

| |

v v

Rules Rules

Apriori Flow

1. Expand DataSources and select ArffLoader.
2. Place ArffLoader and configure the ARFF file.
3. Expand Association → trees and select Apriori.

4. Connect ArffLoader to Apriori using dataSet.
5. Add TextViewer.
6. Connect Apriori to TextViewer.
7. Press Play.
8. Select Show results.

FP-Growth Flow

1. Place ArffLoader and configure the ARFF file.
2. Select FPGrowth from Association → trees.
3. Connect ArffLoader to FPGrowth.
4. Add TextViewer and connect it.
5. Press Play.
6. Select Show results.

Result

Knowledge Flow layouts for Apriori and FP-Growth were successfully developed.

QUESTION 6

ZeroR Technique on Iris Dataset

Aim

To demonstrate ZeroR technique on the Iris dataset using necessary preprocessing techniques and share observations.

Procedure

1. Open WEKA Explorer.
2. Load iris.arff.
3. Select Classify.
4. Click Choose and select ZeroR.
5. Accept default parameters.
6. Select 10-fold cross-validation.
7. Click Start.
8. Observe the model and evaluation statistics.

Observation

ZeroR is used as a basic classification technique and provides a baseline classification result. It does not construct a decision tree from individual input attributes.

Result

The ZeroR classification technique was successfully demonstrated on the Iris dataset.

QUESTION 7

Python Program to Calculate Chi-Square Value

Aim

To write a Python program to calculate the Chi-Square value and report the observation.

Program

```
from scipy.stats import chi2_contingency
data = [[207, 282, 241], [234, 242, 232]]
stat, p, dof, expected = chi2_contingency(data)
alpha = 0.05
print("p value is " + str(p))
```



```
if p <= alpha:
    print('Dependent (reject H0)')
else:
    print('Independent (H0 holds true)')
```

Output

p value is 0.1031971404730939

Independent (H0 holds true)

Observation

Since the p-value is greater than 0.05, the null hypothesis is not rejected.

Result

The Chi-Square value was calculated successfully using Python.

QUESTION 8

Naive Bayesian Classification Using Python

Aim

To write a program of Naive Bayesian classification using Python programming language.

Dataset

The original PDF uses the Wine dataset from Scikit-learn. It contains 13 features and three target classes: class_0,

class_1 and class_2.

alcohol

malic_acid

ash

alcalinity_of_ash

magnesium

total_phenols

flavanoids

nonflavanoid_phenols

proanthocyanins

color_intensity

hue

od280/od315_of_diluted_wines

proline

Program

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
```

wine.data, wine.target,

test_size=0.30, random_state=100

```
)
from sklearn.naive_bayes import GaussianNB
gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred = gnb.predict(X_test)
print(y_pred)
```

Accuracy and Confusion Matrix

```
from sklearn import metrics
print(metrics.accuracy_score(y_test, y_pred))
from sklearn.metrics import confusion_matrix
cm = np.array(confusion_matrix(y_test, y_pred))
cm
```

Output

1.0

```
[[14, 0, 0],
 [ 0,19, 0],
 [ 0, 0,21]]
```


Result

The Naive Bayesian classification program was successfully executed using Python. The accuracy obtained in the original

PDF is 1.0.

QUESTION 9

Java Program to Prepare a Simulated Dataset with Unique Instances

Aim

To write a Java program to prepare a simulated dataset with unique instances.

Program

```
import java.io.*;
public class Demo
{
    static void distinct_vals(int my_arr[], int len)
    {
        for (int i = 0; i < len; i++)
        {
            int j;
            for (j = 0; j < i; j++)
                if (my_arr[i] == my_arr[j])
```

break;

```
            if (i == j)
```

System.out.print(my_arr[i] + " ");

```
        }
    }
```

public static void main (String[] args)

```
    {
        int my_arr[] = {
            55, 67, 99, 11, 54,
            55, 88, 99, 1, 13, 45
        };
        int arr_len = my_arr.length;
```

System.out.println(

"The distinct elements in the array are "

```
        );
```

distinct_vals(my_arr, arr_len);

```
    }
}
```

Output

The distinct elements in the array are

```
55 67 99 11 54 88 1 13 45
```

Observation

Duplicate values such as 55 and 99 occur more than once in the input, but the program prints each distinct value only once.

Result

The Java program successfully prepared and displayed unique instances from the simulated dataset.

EXPERIMENT-15

Aim

Write a program to compute/display dissimilarity matrix (for your own dataset containing at least four instances with two attributes) using Python.

Program

```
import numpy as np
from scipy.spatial import distance_matrix
# Create the matrices
x = np.array([[1,2],[2,1],[2,2]])
y = np.array([[5,0],[1,2],[2,0]])
# Display the matrices
print("matrix x:\\n", x)
print("matrix y:\\n", y)
# compute the distance matrix
dist_mat = distance_matrix(x, y, p=2)
print("Distance Matrix:\\n", dist_mat)
```

Output

```
matrix x:
 [[1 2]
  [2 1]
  [2 2]]
matrix y:
 [[5 0]
  [1 2]
  [2 0]]

Distance Matrix:
 [[4.47213595 0.         2.23606798]
  [3.16227766 1.41421356 1.        ]
  [3.60555128 1.         2.        ]]
```

Result

The program successfully computed and displayed the dissimilarity matrix between the two datasets using the Euclidean distance measure.